

(A Partial) Serializable Snapshot Isolation

Now, we are going to add a conflict detection algorithm at the transaction commit time, so as to make the engine to have some level of serializable.

To run test cases,

```
cargo x copy-test --week 3 --day 6
cargo x scheck
```

Let us go through an example of serializable. Consider that we have two transactions in the engine that:

```
txn1: put("key1", get("key2"))
txn2: put("key2", get("key1"))
```

The initial state of the database is `key1=1, key2=2`. Serializable means that the outcome of the execution has the same result of executing the transactions one by one in serial in some order. If we execute `txn1` then `txn2`, we will get `key1=2, key2=2`. If we execute `txn2` then `txn1`, we will get `key1=1, key2=1`.

However, with our current implementation, if the execution of these two transactions overlaps:

```
txn1: get key2 <- 2
txn2: get key1 <- 1
txn1: put key1=2, commit
txn2: put key2=1, commit
```

We will get `key1=2, key2=1`. This cannot be produced with a serial execution of these two transactions. This phenomenon is called write skew.

With serializable validation, we can ensure the modifications to the database corresponds to a serial execution order, and therefore, users may run some critical workloads over the system that requires serializable execution. For example, if a user runs bank transfer workloads on Mini-LSM, they would expect the sum of money at any point of time is the same. We cannot guarantee this invariant without serializable checks.

One technique of serializable validation is to record read set and write set of each transaction in the system. We do the validation before committing a transaction (optimistic concurrency control). If the read set of the transaction overlaps with any transaction committed after its read timestamp, then we fail the validation, and abort the transaction.

Back to the above example, if we have `txn1` and `txn2` both started at timestamp = 1.

```

txn1: get key2 <- 2
txn2: get key1 <- 1
txn1: put key1=2, commit ts = 2
txn2: put key2=1, start serializable verification

```

When we validate txn2, we will go through all transactions started before the expected commit timestamp of itself and after its read timestamp (in this case, $1 < ts < 3$). The only transaction satisfying the criteria is txn1. The write set of txn1 is `key1`, and the read set of txn2 is `key1`. As they overlap, we should abort txn2.

Task 1: Track Read Set in Get and Write Set

In this task, you will need to modify:

```

src/mvcc/txn.rs
src/mvcc.rs

```

When `get` is called, you should add the key to the read set of the transaction. In our implementation, we store the hashes of the keys, so as to reduce memory usage and make probing the read set faster, though this might cause false positives when two keys have the same hash. You can use `farmhash::hash32` to generate the hash for a key. Note that even if `get` returns a key is not found, this key should still be tracked in the read set.

In `LsmMvccInner::new_txn`, you should create an empty read/write set for the transaction if `serializable=true`.

Task 2: Track Read Set in Scan

In this task, you will need to modify:

```

src/mvcc/txn.rs

```

In this course, we only guarantee full serializability for `get` requests. You still need to track the read set for scans, but in some specific cases, you might still get non-serializable result.

To understand why this is hard, let us go through the following example.

```

txn1: put("key1", len(scan(..)))
txn2: put("key2", len(scan(..)))

```

If the database starts with an initial state of `a=1, b=2`, we should get either `a=1, b=2, key1=2, key2=3` or `a=1, b=2, key1=3, key2=2`. However, if the transaction execution

is as follows:

```
txn1: len(scan(..)) = 2
txn2: len(scan(..)) = 2
txn1: put key1 = 2, commit, read set = {a, b}, write set = {key1}
txn2: put key2 = 2, commit, read set = {a, b}, write set = {key2}
```

This passes our serializable validation and does not correspond to any serial order of execution! Therefore, a fully-working serializable validation will need to track key ranges, and using key hashes can accelerate the serializable check if only `get` is called. Please refer to the bonus tasks on how you can implement serializable checks correctly.

Task 3: Engine Interface and Serializable Validation

In this task, you will need to modify:

```
src/mvcc/txn.rs
src/lsm_storage.rs
```

Now, we can go ahead and implement the validation in the commit phase. You should take the `commit_lock` every time we process a transaction commit. This ensures only one transaction goes into the transaction verification and commit phase.

You will need to go through all transactions with commit timestamp within range `(read_ts, expected_commit_ts)` (both excluded bounds), and see if the read set of the current transaction overlaps with the write set of any transaction satisfying the criteria. If we can commit the transaction, submit a write batch, and insert the write set of this transaction into `self.inner.mvcc().committed_txns`, where the key is the commit timestamp.

You can skip the check if `write_set` is empty. A read-only transaction can always be committed.

You should also modify the `put`, `delete`, and `write_batch` interface in `LsmStorageInner`. We recommend you define a helper function `write_batch_inner` that processes a write batch. If `options.serializable = true`, `put`, `delete`, and the user-facing `write_batch` should create a transaction instead of directly creating a write batch. Your write batch helper function should also return a `u64` commit timestamp so that `Transaction::Commit` can correctly store the committed transaction data into the MVCC structure.

Task 4: Garbage Collection

In this task, you will need to modify:

```
src/mvcc/txn.rs
```

When you commit a transaction, you can also clean up the committed txn map to remove all transactions below the watermark, as they will not be involved in any future serializable validations.

Test Your Understanding

- If you have some experience with building a relational database, you may think about the following question: assume that we build a database based on Mini-LSM where we store each row in the relation table as a key-value pair (key: primary key, value: serialized row) and enable serializable verification, does the database system directly gain ANSI serializable isolation level capability? Why or why not?
- The thing we implement here is actually write snapshot-isolation (see [A critique of snapshot isolation](#)) that guarantees serializable. Is there any cases where the execution is serializable, but will be rejected by the write snapshot-isolation validation?
- There are databases that claim they have serializable snapshot isolation support by only tracking the keys accessed in gets and scans (instead of key range). Do they really prevent write skews caused by phantoms? (Okay... Actually, I'm talking about [BadgerDB](#).)

We do not provide reference answers to the questions, and feel free to discuss about them in the Discord community.

Bonus Tasks

- **Read-Only Transactions.** With serializable enabled, we will need to keep track of the read set for a transaction.
- **Precision/Predicate Locking.** The read set can be maintained using a range instead of a single key. This would be useful when a user scans the full key space. This will also enable serializable verification for scan.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.